Heap

06 May 2026 18:37

What is heap memory?
In Java, the heap is a runtime memory area where:
- Objects are created using **new** keyword
- Instance variables are stored
- Managed by Garbage Collection(gc)

Features of Heap memory
- Shared among threads
- Dynamically allocated
- Automatically managed
- Garbage Collector removes unused objects.

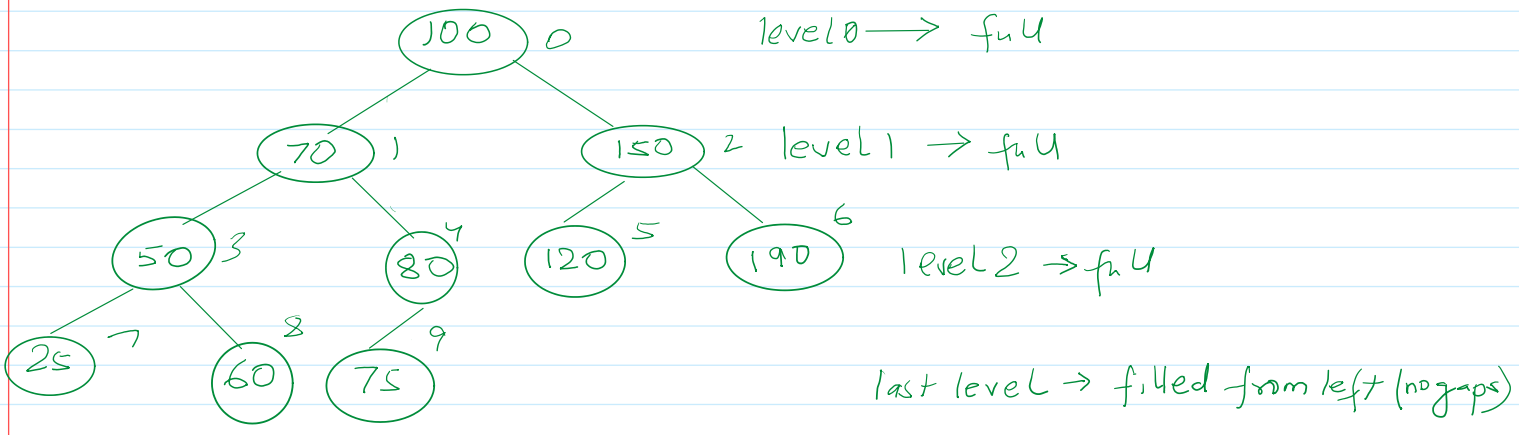
Heap Data Structure

What is heap?
A heap is a complete binary tree that satisfies the heap property.

Types:
1. Max heap---> parent node >= child node
2. Min heap---> parent node <= child node

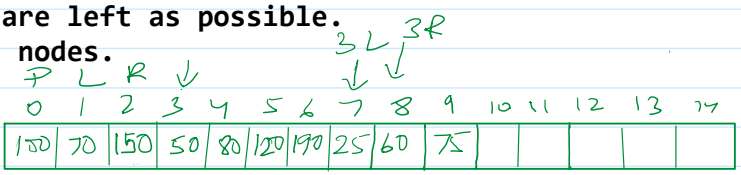
A heap is a complete binary tree(CBT).

What is complete binary tree?
A complete binary tree is a specific type of binary tree with strict structural property.
Definition:
A complete binary tree :
   a. All levels are completely filled except possibly the last level.
   b. The last level is filled from left to right without gaps.



Properties:
1. Notes are filled level by level.
2. Last level nodes are left as possible.
3. No "gaps" between nodes.

1. Nodes are filled level by level.
2. Last level nodes are left as possible.
3. No "gaps" between nodes.



What is the relation with array representation?

Parent = (i − 1) / 2
left child = 2*i + 1
right child = 2*i + 2

} By using this relationship, we can fetch our required information.

Why complete binary tree is important?

It represents:
i. Heap data structure
ii. Heap sort
iii. Priority Queue
iv. Efficient storage(array-based)
v. No gaps -> predictable indexing

Height Property: For a complete binary tree with N nodes:
Height = $\lfloor \log_2(n) \rfloor$

Heap Sort

Heap sort is a comparison based sorting algorithm that:
1. Builds a Max heap
2. Repeatedly extracts the maximum element
3. Place it at the end

To sort an array using heap
1. Convert array into max heap.
2. Swap root (largest) with last element.
3. Reduce the heap size.
4. Heapify again.
5. Repeat

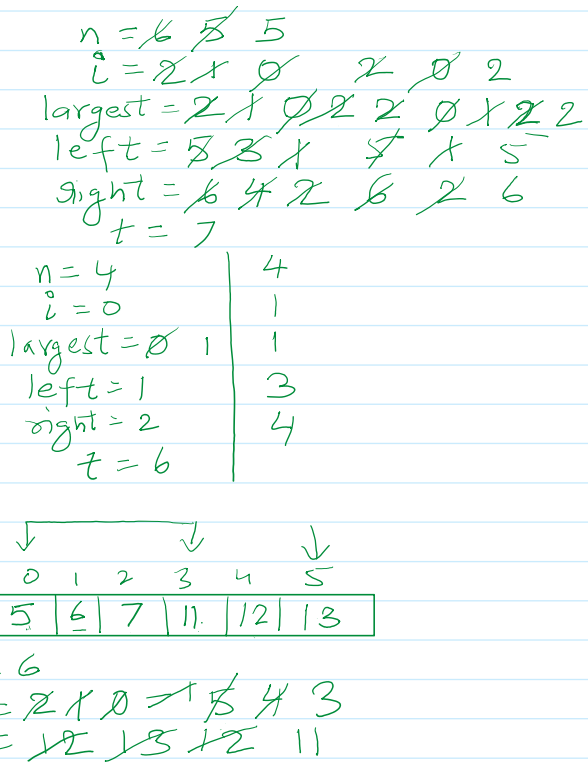
Core Idea

Heapify assumes:
- Left and right are already heaps
- Only the current node may violate the heap property
- So we push down the element to its correct position.

Algorithm
1. Assume current node is largest.
2. Compare with left child.
3. Compare with right child.
4. Swap with largest if needed.
5. Recursively heapify affected subtree.

```java
public static void heapify(int a[], int n, int i){
        int largest = i;
        int left = 2*i + 1;
        int right = 2*i + 2;
        if(left < n && a[left] > a[largest]){
            largest = left;
        }
        if(right < n && a[right] > a[largest]){
            largest = right;
        }
        if(largest != i){
            int t = a[i];
            a[i] = a[largest];
            a[largest] = t;
            heapify(a, n, largest);
        }

    }
    public void sort(int a[]){
        int n = a.length;
        //step 1: Builds a Max heap
        for(int i = n/2 - 1; i > -1; i--){
            heapify(a, n, i);
        }
        //step 2:Repeatedly extracts the maximum element
        for(int i = n - 1; i > 0; i--){
            int t = a[0];
            a[0] = a[i];
            a[i] = t;
            heapify(a, i, 0);
        }
    }
```



Sort the array into descending order, and try to use generics in your code.